

Team Details:

Kadin Raju - u25175956

Tshego - u23605032

Tamlyn Khan - u24675815

System Description:

The project combines four Gang of Four design patterns - Composite, Iterator, State and Decorator - into one integrated application rather than demonstrating each pattern separately.

The Composite pattern represents the nested hierarchy, allowing individual work items and groups of work to be handled through a common abstraction. The Iterator pattern provides different ways of traversing this structure without exposing the internal containers to the client. The State pattern controls lifecycle-dependent behaviour of individual work items, including valid and invalid state transitions. Finally, the Decorator pattern allows additional responsibilities to be attached to suitable objects dynamically at runtime without modifying their original classes.

Together, these patterns allow the system to support nested work structures, independent traversals, changing object behaviour, runtime modification and dynamically added responsibilities.

Composite Hierarchy: Quests are organized into a recursive hierarchy in which individual tasks (**Objective**), quest sequences (**Questline**), and grouped objectives (**QuestGroup**) all implement a common abstract interface (**QuestComponent**). This allows both individual quests and collections of quests to be managed consistently.

State Machine: Quests follow a defined life cycle using **QuestState**. Depending on player progress and interactions, a quest can transition between **LockedState**, **AvailableState**, **ActiveState**, **CompleteState**, and **FailedState**.

Decorator Responsibilities: Additional behaviors and modifiers, such as time limits, reward changes, and difficulty scaling, can be added dynamically at runtime through **QuestDecorator** and its concrete decorators (**ConcreteDecorators**). This extends quest functionality without modifying the underlying quest classes.

Custom Iterators: Quest traversal is handled by custom iterators, including **StoryOrderIterator**, **SingleIterator**, and **AvailableQuestIterator**. These separate traversal logic from the quest data structure, allowing different parts of the quest hierarchy to be navigated according to specific requirements.

Domain and Problem

The chosen domain is the quest and progression system of a story-driven video game. TaskForge models the content a player works through – a Campaign made of Questlines, each made of Quests, each made of individual Objectives – and the journal through which the player views and advances it.

The core difficulty is that quest content is a tree of arbitrary and varying depth, while the code that consumes it wants to treat it as one thing. Printing the quest log, totalling a reward, counting outstanding quests and asking whether a branch is finished are all whole-tree questions. Written against the concrete types, each of those becomes a chain of type tests and special cases at every level, and every new kind of container forces the client to change again.

Three further things vary independently of the tree itself, and it is the combination that makes a plain class hierarchy collapse:

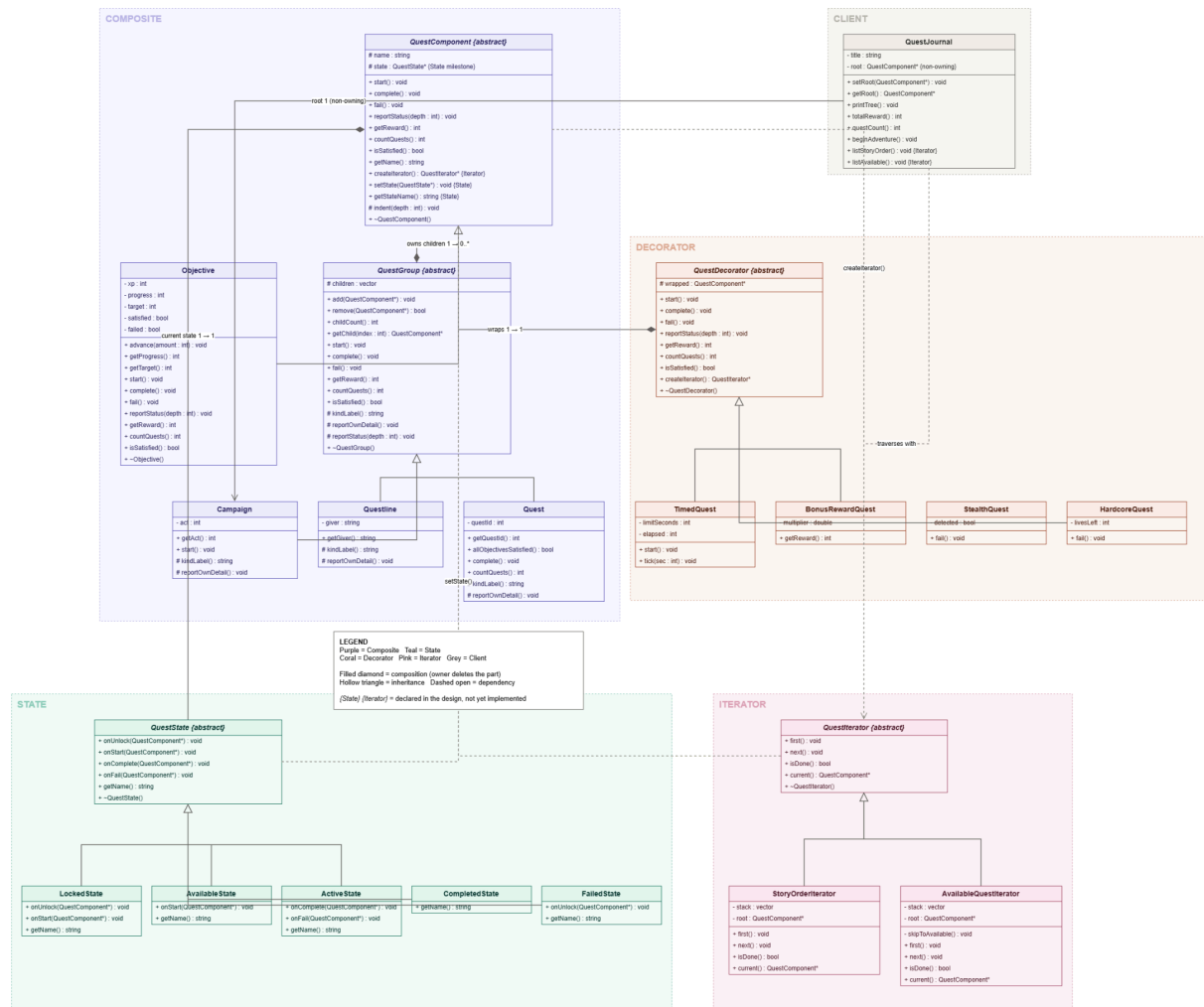
First, every quest moves through a lifecycle – locked until its prerequisites are met, then available, active, and finally complete or failed – and the same call means different things in each phase. Starting an available quest activates it; starting a locked one must be refused. Encoding that as conditionals inside each quest spreads the same branching across every operation.

Second, individual quests carry optional rules: a time limit, a bonus reward, a stealth requirement, a hardcore one-life restriction. Any combination can apply to any quest, so creating a subclass per combination multiplies out of control, and the designer wants to attach these while the game is running rather than at compile time.

Third, the player browses the same content in more than one order: the full story order for the quest log, and only what is currently available for the “what can I do now” view. Both must work without handing the client the internal child collection to walk itself.

The four patterns below each answer one of these pressures, and they compose because all of them are expressed through the single QuestComponent interface

TaskForge — Video Game Quest System : UML Class Diagram



GoF Mapping

Composite

Participant	Class
Component	QuestComponent
Leaf	Objective
Composite	QuestGroup
ConcreteComposite	Quest Questline Campaign
Client	QuestJournal

Ownership Rationale

QuestGroup owns every child passed to add(); its destructor deletes each child still held, so deleting the root of a tree (a Campaign) releases the whole subtree exactly once, top down. remove() releases a child without deleting it - ownership passes back to the caller, who becomes responsible for it from that point on.

QuestJournal is the client that walks the tree but never owns it: it holds root only as a non-owning pointer, set via setRoot(). main() creates and destroys the Campaign; QuestJournal's destructor deletes nothing.

This is the “safety” variant of Composite: add() / remove() / getChild() live on QuestGroup, not on QuestComponent, so a leaf (Objective) cannot be asked to add a child that mistake is caught at compile time instead of needing a runtime check.

Decorator

Participant	Class
Component	QuestComponent
ConcreteComponent	Quest Objective Campaign QuestGroup
Decorator	QuestDecorator
ConcreteDecorator	TimedQuest BonusRewardQuest StealthQuest HardcoreQuest

Ownership Rationale

The QuestDecorator owns the QuestComponent that it wraps. When the outer decorator is destroyed, it destroys the object that it owns. The destruction then continues through the decorator chain.

Iterator

Participant	Class
Iterator	QuestIterator

ConcreteIterator	StoryOrderIterator AvailableQuestIterator
Aggregate	QuestComponent
ConcreteAggregate	QuestGroup Quest Campaign Objective

Traversal Modification Policy

If an iterator is already traversing the structure when a run time change happens, its position may no longer represent the updated structure correctly. The iterator could potentially skip an object, visit an object twice, or refer to an invalid position.

Our policy: an active iterator is not used after the structure it is traversing has been structurally changed. The old iterator is discarded and a new iterator is created for the updated hierarchy.

Traversal-Modification Policy and Important Design Decisions

Traversal-Modification Policy

Our team selected a snapshot traversal policy.

When an iterator is created, it records the objects that belong to that traversal at that point in time. If the Composite structure is subsequently modified - for example, if a work item is added, removed or moved between groups - the iterator that is already in progress continues using its original snapshot.

This means that structural modifications do not unexpectedly change the traversal halfway through its execution. A newly created iterator after the modification will reflect the updated structure.

We selected this policy because it gives predictable iterator behaviour and prevents changes to the Composite hierarchy from invalidating traversal state. It also allows two iterators to traverse the same hierarchy independently, because each iterator maintains its own traversal state.

For example:

Iterator A begins traversing the hierarchy.

A work item is added or moved while Iterator A is active.

Iterator A continues with the structure it originally captured.

Iterator B is created afterwards.

Iterator B sees the new structure.

This makes runtime structural changes safe and gives a clear definition of what an "active traversal" means.

Encapsulation of the Composite

The internal collection of children belonging to a Composite object is not exposed directly to main.cpp. Client code instead requests an appropriate Iterator. This keeps knowledge of how objects are stored and traversed inside the hierarchy rather than coupling the client to an STL container.

Independent Iterator State

Traversal information is stored inside each Iterator rather than globally inside the Composite. Therefore, two iterators can traverse the same hierarchy at different positions without interfering with one another.

Ownership and Object Lifetime

Owning relationships are kept explicit. Composite/group objects are responsible for the lifetime of objects that they own, while temporary traversal objects do not take ownership of the underlying work items. Polymorphic base classes use virtual destructors so that derived objects can be destroyed correctly through base-class pointers.

This division of ownership reduces the risk of double deletion and memory leaks.

State Behaviour

Lifecycle behaviour is delegated to State objects rather than implemented using one large switch statement in the work-item class. Each State determines which operations and transitions are valid in that stage of the object's lifecycle.

Invalid transitions are handled explicitly rather than silently placing an object into an invalid state.

Decorator Behaviour

Decorators preserve the same relevant interface as the objects they wrap. This means decorated and undecorated objects can participate in the normal system in the same way.

Decorators can also be stacked, allowing multiple optional responsibilities to be combined at runtime without creating subclasses for every possible combination.

Ownership Rationale

The iterator does not own the quest objects that it visits. The QuestGroup owns its children. The iterator only temporarily refers to those objects while it performs traversal, so the iterator must not delete the objects it visits. This prevents double deletion and keeps ownership in one clear place. The iterator can therefore be destroyed after traversal without destroying the quests themselves.

State

Participant	Class
Context	QuestComponent
State	QuestState
ConcreteState(s)	LockedState AvailableState FailedState ActiveState CompleteState

The State pattern is used to handle the different behaviour of a quest depending on its current state. Instead of putting all the state checks inside the Quest class, each state handles its own behaviour. For example, when a quest is active and the player completes it, the quest changes to the Complete state. If the quest is still locked, completing it has no effect.

This makes the system easier to manage because each state is responsible for deciding what should happen when an action is performed.

Invalid State Changes

Some actions are not allowed depending on the current state of the quest. When this happens, nothing happens rather than causing an error. For example, a locked quest cannot be started, an active quest cannot be started again, and a completed quest cannot be failed... This helps prevent the quest from changing to a state that it should not be in.

Starting a Quest

When a quest is started, the system checks that the quest has actually changed to the Active state. If it has, the start action is passed down to the objectives belonging to that quest. This resets the objectives so that their progress starts from the beginning.

The check is important because the objectives should only be reset when the quest actually starts. Trying to start a quest that is still locked should not reset its objectives.

Retrying a Failed Quest

A failed quest can be attempted again. When the player unlocks a failed quest, it changes back to the Available state. The player can then start the quest again without needing to create a new quest.

Quest Constructor

When a new quest is created, it starts in the Locked state. This gives every quest a defined starting point. The player must first unlock the quest before they can start it.

GDB and Valgrind

Program state change using GDB:

```
Reading symbols from ./taskforge...
(gdb) forgeBladeRaw->complete();
Undefined command: "forgeBladeRaw->complete". Try "help".
(gdb) break Quest::complete
Breakpoint 1 at 0x4f63: file Quest.cpp, line 30.
(gdb) run
Starting program: /mnt/c/Users/kadin/Desktop/Prac4/taskforge
```

```
Breakpoint 1, Quest::complete (this=0x55555576530) at Quest.cpp:30
30     void Quest::complete() {
(gdb) print this
$1 = (Quest * const) 0x55555576530
(gdb) print this->getName()
$2 = "Forge Blade"
(gdb) print this->getStateName()
$3 = "State: Active"
(gdb) next
31     if (state == 0) return;
(gdb) next
33     QuestState* oldState = state;
(gdb) next
35     state->onComplete();
(gdb) next
40     if (state != oldState && getStateName() == "State: Complete") {
(gdb) print this->getStateName()
$4 = "State: Complete"
(gdb) █
```

Bug using GDB:


```

Reading symbols from ./taskforge...
(gdb) break HardcoreQuest::fail
Breakpoint 1 at 0x44a4: file ConcreteDecorators.cpp, line 63.
(gdb) run
Starting program: /mnt/c/Users/kadin/Desktop/Prac4/taskforge

```

```

Breakpoint 1, HardcoreQuest::fail (this=0x555555778f0)
at ConcreteDecorators.cpp:63
63     livesLeft--;
(gdb) print livesLeft
$1 = 2
(gdb) print livesLeft
$2 = 2
(gdb) next
65     if (livesLeft < 0) {
(gdb) print livesLeft
$3 = 1
(gdb) next
68     }
(gdb) print livesLeft
$4 = 1
(gdb) continue
Continuing.
defeatBanditsHard state after 1st hit: State: Active

Breakpoint 1, HardcoreQuest::fail (this=0x555555778f0)
at ConcreteDecorators.cpp:63
63     livesLeft--;
(gdb) print livesLeft
$5 = 1
(gdb) next
65     if (livesLeft < 0) {
(gdb) print livesLeft
$6 = 0
(gdb) print livesLeft < 0
$7 = false
(gdb) print livesLeft <= 0
$8 = true

```

During development, the HardcoreQuest decorator did not propagate failure when the player's final life was lost. The symptom was that after two failures, the Defeat Bandits quest remained Active instead of changing to Failed.

I used GDB to investigate the HardcoreQuest::fail() function. A breakpoint was placed at HardcoreQuest::fail() and the value of livesLeft was inspected while stepping through the function. On the second failure, livesLeft changed from 1 to 0, but the condition controlling failure was `livesLeft < 0`. Since zero is not less than zero, `wrapped->fail()` was not called.

The bug was corrected by changing the condition to `livesLeft <= 0`. After recompiling and rerunning the program, the second failure correctly propagated to the wrapped quest and its state changed to Failed.

Valgrind

```

==1155==
==1155== HEAP SUMMARY:
==1155==   in use at exit: 0 bytes in 0 blocks
==1155== total heap usage: 120 allocs, 120 frees, 78,346 bytes allocated
==1155==
==1155== All heap blocks were freed -- no leaks are possible
==1155==
==1155== For lists of detected and suppressed errors, rerun with: -s
==1155== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
kadin@KADINPC: /mnt/c/Users/kadin/Desktop/Prac4$

```

GitHub Repository Link and Workflow Reflection

GitHub Repository:

<https://github.com/u25175956-ui/214-Prac4>

TamlynKhan = Tamlyn Khan; zandil3 = Tshegofatso Kungwane; u25175956-ui = Kadin Raju
GitHub was used throughout the development of TaskForge to divide the required design-pattern work between the three team members and then integrate the components into one complete system. The repository history shows separate contributions for the Composite, State, Decorator and Iterator implementations, rather than the project being uploaded only at the end.

The work was divided according to major project components. TamlynKhan worked on the initial Composite design and implementation, helped configure the Makefile and Docker environment, and later added the final UML material. zandil3 implemented the State pattern, including the QuestState hierarchy and its concrete states. u25175956-ui implemented the Decorator pattern and subsequently the Iterator pattern, including the concrete traversal implementations. This division corresponds directly with the major GoF areas in the final system; for example, the design identifies QuestIterator with StoryOrderIterator and AvailableQuestIterator as its concrete iterators.

The workflow allowed each team member to concentrate on a major responsibility while still contributing to the same shared codebase. As the patterns depended on common classes such as QuestComponent, QuestGroup, Quest and Campaign, integration required the team to keep interfaces consistent when new functionality was added. The Iterator implementation in particular had to work with the existing Composite hierarchy without taking ownership of the quest objects, which is consistent with the final ownership design.

Team Contribution Statement:

Kadin also created testing main and did the Valgrind and GDB. Tsehgo also did the five diagrams. Tamlyn wrote up parts of the pdf and compiled the GitHub contributions etc.

The rest of the summary of tasks is here:

Team member	Main contribution
TamlynKhan	Developed the Composite design/implementation , assisted with the Makefile and Docker setup , and added the final UML design material .
zandil3	Implemented the State pattern , including <code>QuestState</code> , <code>LockedState</code> , <code>AvailableState</code> , <code>ActiveState</code> , <code>CompleteState</code> and <code>FailedState</code> .
u25175956-ui	Implemented the Decorator pattern , including <code>QuestDecorator</code> and its concrete decorators, and implemented the Iterator pattern , including the iterator hierarchy and traversal behaviour.

TaskForge Quest System

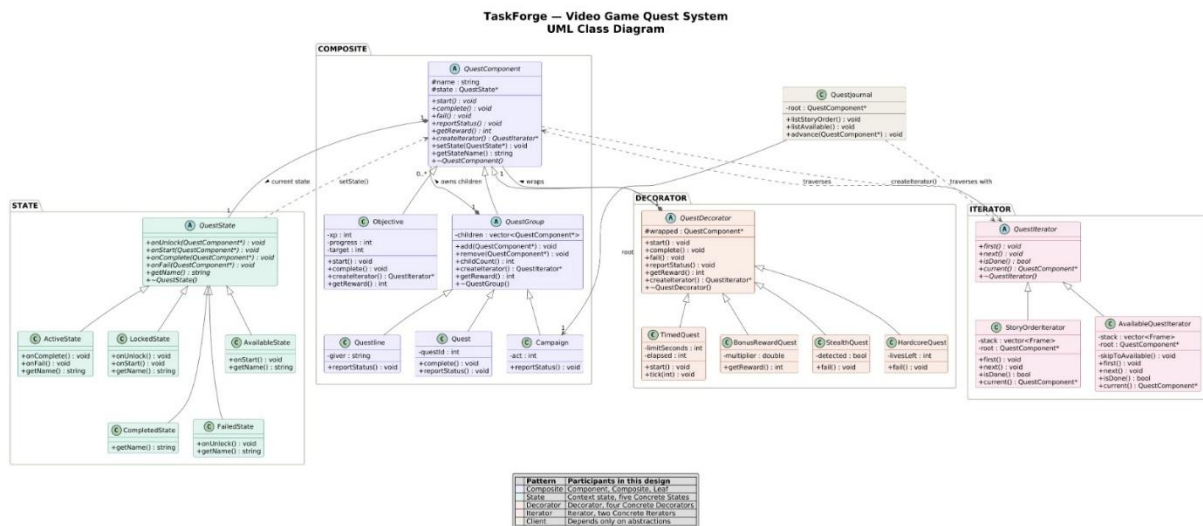
UML & Workflow Documentation

Design Patterns, Runtime Structure, State Lifecycle, and Activity Workflows

Four GoF Patterns Integrated:

Composite • State • Decorator • Iterator

Diagram 1: UML Class Diagram



Overview

The design integrates four GoF patterns around a single QuestComponent abstraction:

Composite Pattern

The hierarchy is rooted at QuestComponent (Component interface). QuestGroup is the Composite that owns vector<QuestComponent*> children. The concrete Composites are Campaign (root), Questline (middle), and Quest (lowest). Objective is the Leaf with no children. All recursive operations (start(), complete(), fail(), getReward(), isSatisfied(), countQuests()) are delegated from parent to children.

State Pattern

Each QuestComponent holds a QuestState* state pointer (making it the Context). QuestState is the abstract State interface. Concrete states are LockedState, AvailableState, ActiveState, CompleteState, and FailedState. State transitions happen when a lifecycle method (e.g., onStart(), onComplete()) calls quest->setState(newState) on the context.

Decorator Pattern

QuestDecorator extends QuestComponent and wraps another QuestComponent* wrapped (composition, not inheritance). Each decorator delegates all operations to its wrapped component and adds one specific behavior. Concrete decorators are TimedQuest (enforces time limits), BonusRewardQuest (multiplies rewards),

StealthQuest (conditional failure), and HardcoreQuest (absorbs failures). Multiple decorators can be stacked on a single quest at runtime.

Iterator Pattern

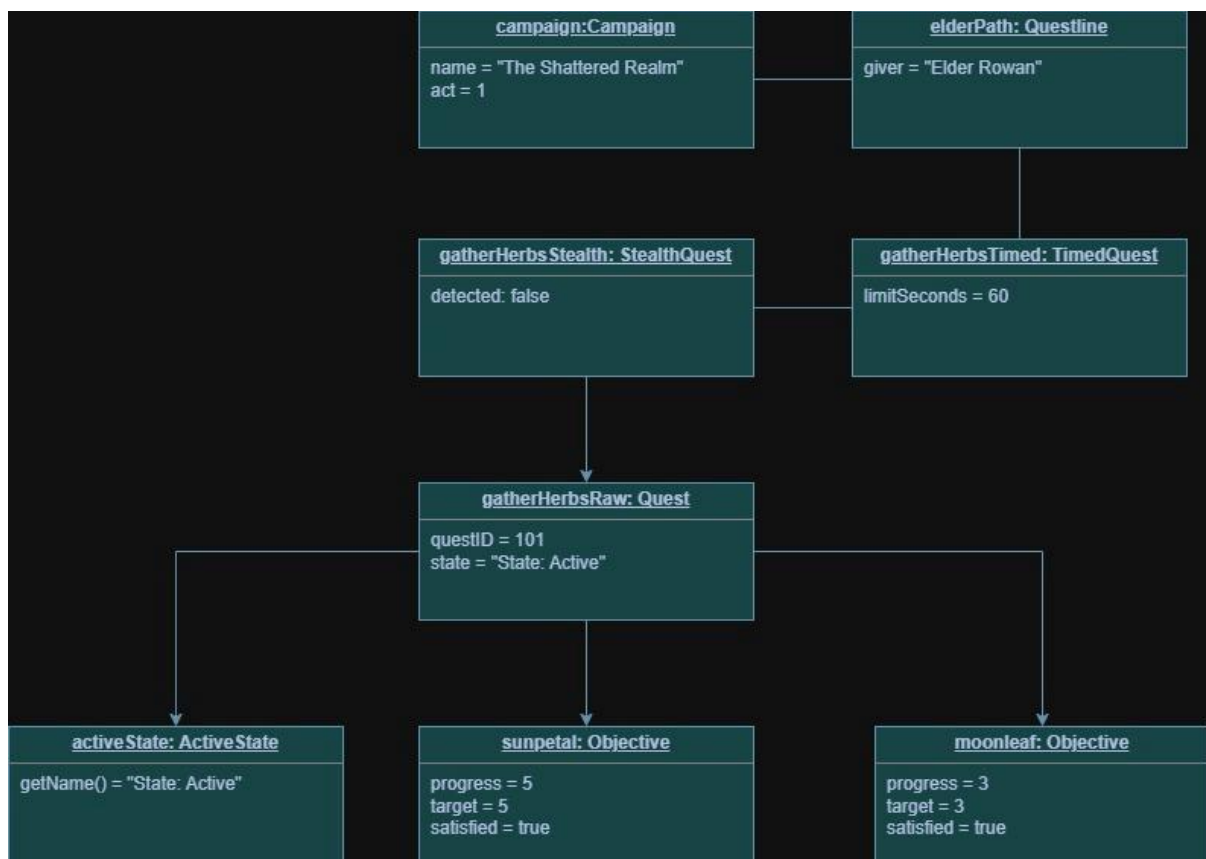
QuestIterator is the abstract iterator with methods first(), next(), isDone(), and current(). StoryOrderIterator does recursive depth-first traversal using nested child iterators. SingleIterator handles leaves (Objectives).

AvailableQuestIterator filters traversal to only State: Available components. Iterators encapsulate traversal logic; client code never calls getChild() directly.

Key Integration

A single pointer type (QuestComponent*) allows the client (QuestJournal) to work with Campaigns, Questlines, Quests, Objectives, and any decorated variant identically—client code cannot tell them apart and never needs to.

Diagram 2: UML Object Diagram (Runtime Snapshot)



Overview

This object diagram captures a meaningful runtime state partway through gameplay:

The Snapshot Structure

campaign: Campaign ("The Shattered Realm", act=1)

-> elderPath: Questline (giver="Elder Rowan")

-> gatherHerbsTimed: TimedQuest (limitSeconds=60)

-> gatherHerbsStealth: StealthQuest (detected=false)

-> gatherHerbsRaw: Quest (questId=101)

-> activeState: ActiveState (getName() = "State: Active")

- > sunpetal: Objective (progress=5/5, satisfied)
- > moonleaf: Objective (progress=3/3, satisfied)

Pattern Visibility

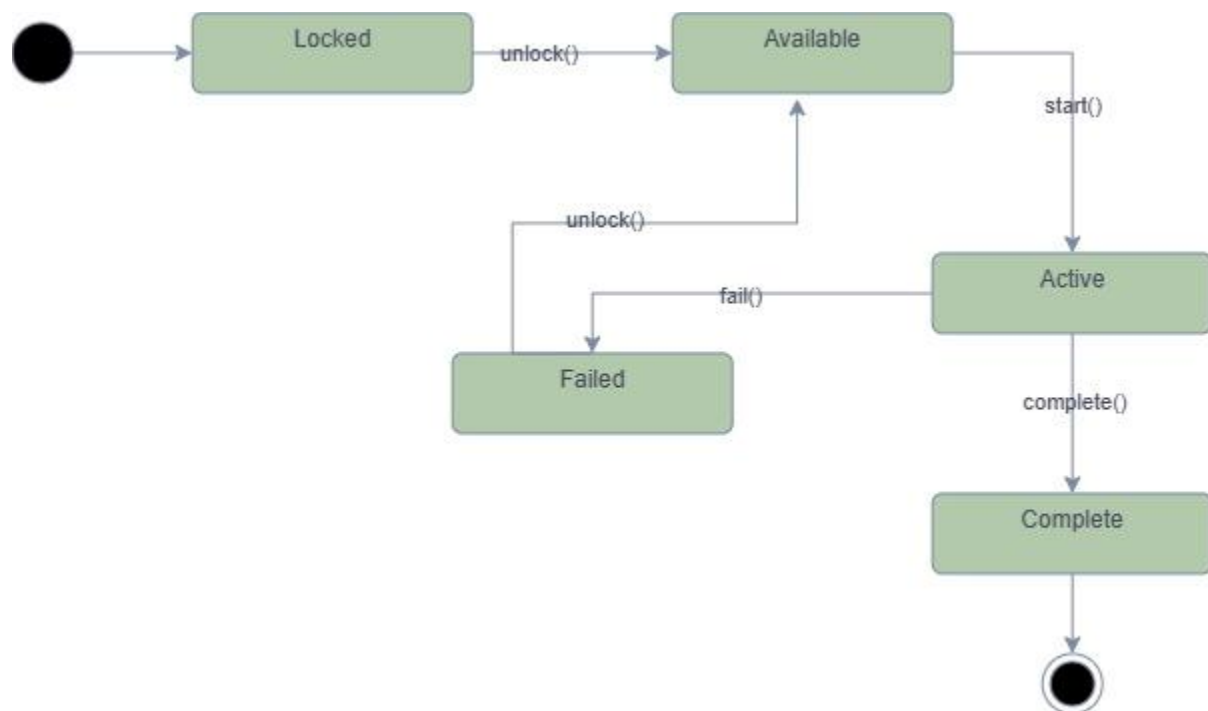
Composite: The tree structure shows parent-child relationships (aggregation). Each QuestComponent holds zero or more children of type QuestComponent*. Objectives are leaves; Quests and Questlines are composites.

State: Each Quest object holds a reference to its current state object. For example, gatherHerbsRaw.state points to an ActiveState instance. This enables polymorphic behavior: calling gatherHerbsRaw->complete() delegates to the state's onComplete(), which transitions the quest based on its current mode.

Decorator: The decorator chain is visible as nested wrapped pointers. gatherHerbsTimed wraps gatherHerbsStealth, which wraps gatherHerbsRaw. Each decorator delegates to the next layer. When getReward() is called on gatherHerbsTimed, it eventually queries gatherHerbsRaw, but each layer can modify the result.

Iterator: Not visible in this static snapshot, but iterators would walk this tree structure. A StoryOrderIterator starting at campaign would visit nodes in depth-first order; an AvailableQuestIterator would skip any quest not in State: Available.

Diagram 3: UML State Diagram (Quest Lifecycle)



Overview

This state machine diagram models the lifecycle of a Quest object (the Context in the State pattern):

States and Their Meaning

Locked: The quest exists but the player has not yet earned the right to accept it. Calling start(), complete(), or fail() has no effect. Only unlock() transitions to Available.

Available: The quest is unlocked and the player can accept it. Calling start() transitions to Active. All other operations have no effect.

Active: The quest is running. The player is making progress on its objectives. Calling `complete()` transitions to Complete. Calling `fail()` transitions to Failed.

Complete: Terminal state. The quest succeeded. All operations have no effect.

Failed: Terminal state. The quest failed. Calling `unlock()` transitions back to Available (giving the player a retry).

Transitions and Events

`quest->unlock()`: Locked → Available (or no-op in other states)

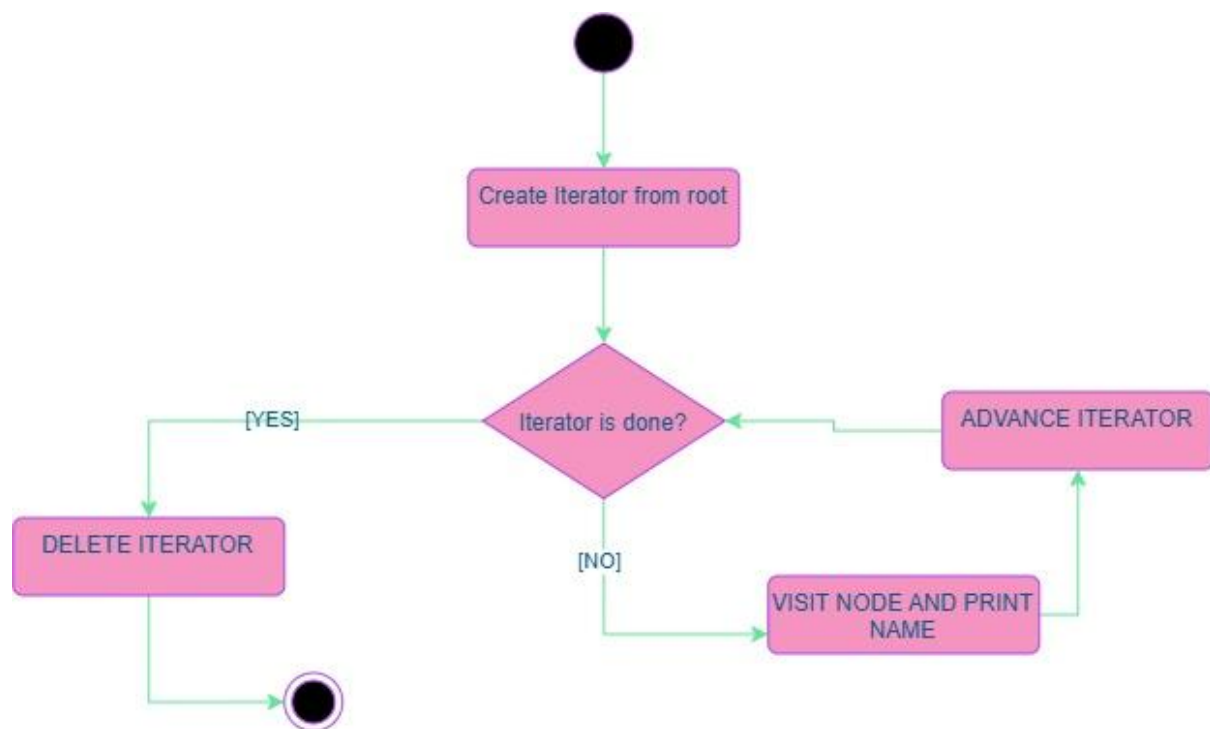
`quest->start()`: Available → Active (or no-op in other states)

`quest->complete()`: Active → Complete (or no-op in other states)

`quest->fail()`: Active → Failed (or no-op in other states)

Implementation Detail: Invalid transitions are handled gracefully by implementing the do nothing behavior in each state. For example, `LockedState::onComplete()` is empty—it does not throw an exception. This is the safety variant: the state object knows what it should do in response to each event.

Diagram 4: Activity Diagram (Iterator Traversal Workflow)



Overview

This activity diagram makes the traversal mechanism visible as part of a larger workflow, demonstrating how the Iterator pattern encapsulates tree navigation:

The Workflow

1. **Initialize:** The client (e.g., `QuestJournal::listStoryOrder()`) calls `root->createIterator()`, which returns a new `StoryOrderIterator` pointing at the root.
2. **Loop Entry:** The workflow enters a loop by calling `iterator->first()`, which initializes the iterator to point at the first node in traversal order.

3. Loop Body (repeats):

Decision: Check iterator->isDone(). If true, exit loop. If false, continue.

Process: Call iterator->current() to retrieve the current QuestComponent* node and process it.

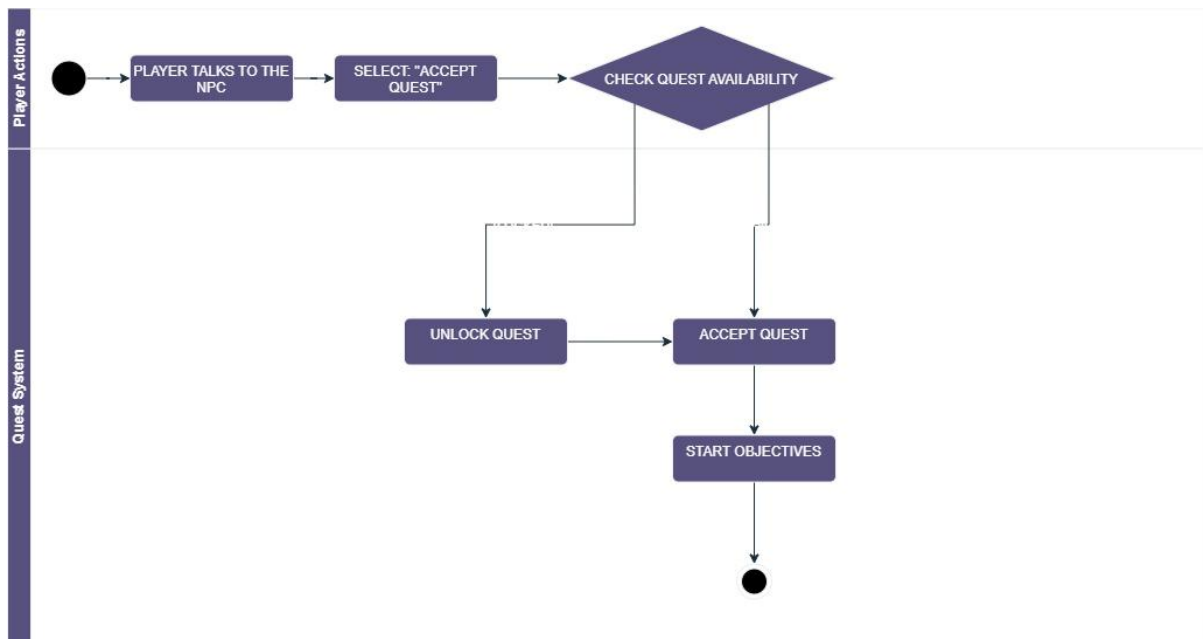
Advance: Call iterator->next() to move to the next node in traversal order.

4. Loop Exit: When isDone() returns true, the workflow ends and the iterator is deleted.

Key Property: The client code never calls getChild(), never manages a vector<>, and never recurses explicitly. The iterator hides the tree structure entirely.

Alternative Traversal: AvailableQuestIterator uses the same loop interface but filters nodes. It wraps a StoryOrderIterator and skips any node whose getStateName() is not State: Available.

Diagram 5: Activity Diagram (Quest Acceptance Workflow)



Overview

This activity diagram models the workflow when a player accepts a quest, using swimlanes to show the two principal actors (Player and Quest System):

Player Swimlane (initiates the action)

1. Player talks to an NPC.
2. Player selects Accept Quest from the dialogue.
3. The action passes to the Quest System swimlane.

Quest System Swimlane (processes the action)

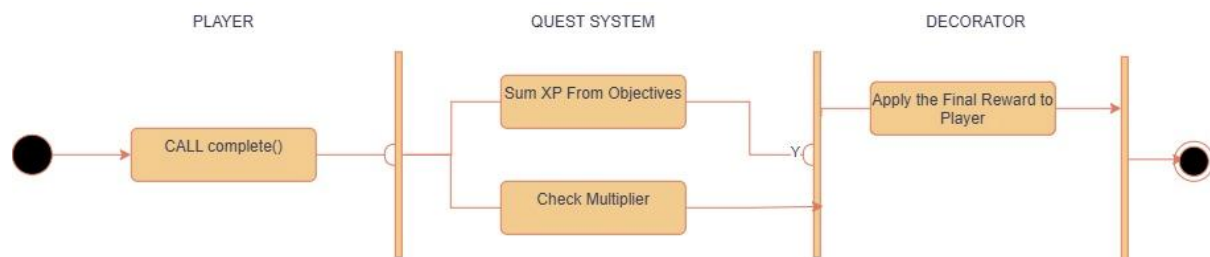
1. Receive the Accept Quest request.
2. Decision: Is the quest in Locked state or Available state?
 - If Locked: Execute Unlock Quest → transitions quest from Locked to Available.
 - If Available: Skip unlock (quest is already available).
3. Both paths merge at Accept Quest.

4. Execute Start Quest → call quest->start(), which transitions to Active and cascades to child objectives.

Workflow Semantics

A locked quest cannot be started directly; the player must unlock it first. Once available, starting a quest immediately transitions it to active and cascades the start action through all descendants.

Diagram 6: Activity Diagram (Completion & Reward Calculation)



Overview

This activity diagram models the quest completion workflow, showing how the Composite and Decorator patterns collaborate in calculating the final reward:

The Workflow

1. Initiate: Player calls quest->complete() (or the game triggers it).
2. State Transition: The quest's current state is asked to handle completion. For a quest in Active state, `ActiveState::onComplete()` transitions the quest to Complete state.
3. Conditional Execution: The state transition is verified. Only if the state actually changed AND the quest is now in Complete state does the following execute:
 - Path 1 (Composite Recursion): Call `QuestGroup::complete()`, which recursively calls `complete()` on every child. Each Objective marks itself complete and triggers reward calculation.
 - Path 2 (Decorator Evaluation): Call `getReward()` on the quest. For a decorated quest (e.g., `BonusRewardQuest`), the decorator multiplies the base reward by its multiplier.
4. Synchronize: Both paths conclude. The quest's total XP is now known and is awarded to the player.

Composite Contribution

The tree structure allows recursive completion: calling `complete()` on a Quest automatically completes all child Objectives. Each Objective returns its XP value only if satisfied. The Quest sums these values via its overridden `getReward()` method.

Decorator Contribution

A decorated quest (e.g., `BonusRewardQuest(quest, 1.5)`) intercepts the `getReward()` call and multiplies by its modifier before returning. If the quest is wrapped in multiple decorators, each layer can apply its own logic.